

LangChain Debugging — Worked Example

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

One scenario, followed across all 4 rounds: Doc05's `chain.py` — the reusable `prompt | model | parser` extraction chain later documents build on. Every round is a different way the chain's *input* doesn't match what the chain actually expects. Work through the rounds in order. Each ends with **"What do you think is wrong?"** — stop and actually answer before reading on.

- [Round 1: Basic](#)
- [Round 2: Intermediate](#)
- [Round 3: Real-world](#)
- [Round 4: Multi-agent](#)

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

Round: Basic

Setup:

```
prompt = ChatPromptTemplate.from_template("Answer this question: {question}")
chain = prompt | ChatOpenAI() | StrOutputParser()
```

Called from another part of the app:

```
result = chain.invoke({"input": "What is the refund policy?"})
```

Symptoms: fails on every call, immediately, before the model is ever reached — not intermittent.

Error output:

```
Traceback (most recent call last):
  File "project_1_supportdesk_chat_and_triage/qa.py", line 18, in answer
    result = chain.invoke({"input": "What is the refund policy?"})
    ...
KeyError: "Input to ChatPromptTemplate is missing variables {'question'}. Expect
```

Expected vs. actual: - Expected: `chain.invoke(...)` fills in the prompt template and returns the model's text answer. - Actual: it fails before ever calling the model — the prompt template can't find the value it needs.

What do you think is wrong?

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

Round: Intermediate

Setup: the invoke-key bug is fixed. The chain now uses a structured Pydantic parser instead of plain text:

```
class Answer(BaseModel):
    summary: str
    confidence: float

chain = prompt | ChatOpenAI() | PydanticOutputParser(pydantic_object=Answer)
```

Symptoms: this looks like it could be an OpenAI API problem (the error mentions "output"), but the API call itself succeeds every time — the model responds, and responds with something that even *looks* like the right JSON to a human reading it.

Error output:

```
Traceback (most recent call last):
  File "project_1_supportdesk_chat_and_triage/qa.py", line 24, in answer
    result = chain.invoke({"question": "What is the refund policy?"})
    ...
langchain_core.exceptions.OutputParserException: Invalid json output: ```json
{"summary": "Refunds are accepted within 30 days.", "confidence": 0.9}
```

The model's actual reply, in full:

```
~~~
```json
{"summary": "Refunds are accepted within 30 days.", "confidence": 0.9}
```

~~~

**Expected vs. actual:** - Expected: `PydanticOutputParser` reads the model's reply and returns a checked `Answer` object. - Actual: the model's answer is genuinely correct JSON — just wrapped in a markdown code fence — and the parser fails on it anyway.

### What do you think is wrong?

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

## Round: Real-world

**Setup:** the markdown-fence bug is fixed. The chain works reliably in testing with short questions. In production, `question` sometimes comes from pasted user content, which occasionally contains its own curly braces (a snippet of JSON, a config example, code the user is asking about).

**Symptoms:** only sometimes happens — depends entirely on what text is in the specific question being asked, not on load or timing.

**Error output (only for certain inputs):**

```
Traceback (most recent call last):
 File "project_1_supportdesk_chat_and_triage/qa.py", line 24, in answer
 result = chain.invoke({"question": user_question})
 ...
KeyError: 'status'
```

The failing `user_question` was: `Why does my config return {"status": "error"} instead of 200?`

**Expected vs. actual:** - Expected: any question text, including text that happens to contain `{` and `}`, gets substituted into the prompt safely. - Actual: works perfectly for ordinary questions, and fails specifically when the question text itself contains curly braces — which look, to the template engine, like more variables to fill in.

**What do you think is wrong?**

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

## Round: Multi-agent

**Setup:** the curly-brace bug is fixed for direct user input. Separately, in Project 4, the Analysis agent takes the Research agent's findings (plain text, written into shared state) and feeds them into this same LCEL chain as its own `{question}` -equivalent input, to double-check a specific claim.

**Symptoms:** testing the chain directly with hand-typed inputs never triggers this. Running the full pipeline, it fails only on certain requests — specifically ones where the Research agent's search results happened to include a JSON snippet (an API response it quoted, a config example it found).

### Error output:

```
[research_agent] research_findings written to shared state (excerpt):
"...the API returned {"error": "rate_limited"} on the first attempt..."

Traceback (most recent call last):
 File "agents/analysis_agent.py", line 19, in verify_claim
 result = verification_chain.invoke({"claim_context": research_findings})
 ...
KeyError: 'error'
```

**Expected vs. actual:** - Expected: whatever text the Research agent found, including any JSON-shaped snippets inside it, passes safely into the Analysis agent's own chain. - Actual: it fails only when Research's findings happen to contain literal curly braces — invisible testing the Analysis agent alone with clean, hand-written findings, because nobody hand-writing a test input thinks to include a stray JSON snippet.

### What do you think is wrong?

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Solution](#)

Full solution: [Show me the solution](#)